

Practical / Journal Assignment - 02 | SOLVED ANSWERS

Q1. Create a Student Registration Form using HTML Helpers (TextBox, TextArea, DropDownList, RadioButton, CheckBox, Password, Hidden).

Theory: HTML Helpers are C# methods that generate HTML markup. They are called inside Razor views using `@Html.MethodName()`. They provide a server-side way to create HTML form elements.

Models/StudentModel.cs

```
public class StudentModel
{
    public int StudentId { get; set; }
    public string Name { get; set; }
    public string Address { get; set; }
    public string Course { get; set; }
    public string Gender { get; set; }
    public bool IsSports { get; set; }
    public bool IsReading { get; set; }
    public bool IsCoding { get; set; }
    public string Password { get; set; }
}
```

Controllers/StudentController.cs

```
public class StudentController : Controller
{
    [HttpGet]
    public IActionResult Register()
    {
        ViewBag.Courses = new List<SelectListItem>
        {
            new SelectListItem { Value="BCA", Text="BCA" },
            new SelectListItem { Value="BSc CS", Text="BSc CS" },
            new SelectListItem { Value="BSc ECS", Text="BSc ECS" }
        };
        return View(new StudentModel { StudentId = 101 });
    }

    [HttpPost]
    public IActionResult Register(StudentModel model)
    {
        TempData["Msg"] = $"Registered: {model.Name}, Course: {model.Course}";
        return RedirectToAction("Register");
    }
}
```

Views/Student/Register.cshtml

```
@model StudentModel
@{ ViewData["Title"] = "Student Registration"; }

<h2>Student Registration Form</h2>
@using (Html.BeginForm("Register", "Student", FormMethod.Post))
{
    @* Hidden Field *@
```

```

@Html.Hidden("StudentId", Model.StudentId)

<div class="mb-2">
    <label>Name:</label>
    @Html.TextBox("Name", null, new { @class="form-control", placeholder="Enter Name" })
</div>

<div class="mb-2">
    <label>Address:</label>
    @Html.TextArea("Address", "", new { @class="form-control", rows=3 })
</div>

<div class="mb-2">
    <label>Course:</label>
    @Html.DropDownList("Course", (IEnumerable<SelectListItem>)ViewBag.Courses,
        "-- Select Course --", new { @class="form-select" })
</div>

<div class="mb-2">
    <label>Gender:</label>
    @Html.RadioButton("Gender", "Male") Male
    @Html.RadioButton("Gender", "Female") Female
</div>

<div class="mb-2">
    <label>Hobbies:</label>
    @Html.CheckBox("IsSports") Sports
    @Html.CheckBox("IsReading") Reading
    @Html.CheckBox("IsCoding") Coding
</div>

<div class="mb-2">
    <label>Password:</label>
    @Html.Password("Password", null, new { @class="form-control" })
</div>

<button type="submit" class="btn btn-primary">Register</button>
}

@if (TempData["Msg"] != null)
{
    <div class="alert alert-success mt-3">@TempData["Msg"]</div>
}

```

Q2. Create a Custom HTML Helper for generating a styled button.

Theory: Custom HTML Helpers are extension methods on the `HtmlHelper` class. They allow you to create reusable, custom HTML-generating methods to use in Razor views.

Helpers/CustomHtmlHelper.cs

```

using Microsoft.AspNetCore.Html;
using Microsoft.AspNetCore.Mvc.Rendering;

public static class CustomHtmlHelper
{
    // Custom Styled Button Helper
    public static IHtmlContent StyledButton(
        this IHtmlHelper htmlHelper,
        string text,
        string color = "primary",

```

```

        string size = "md")
    {
        var button = new TagBuilder("button");
        button.AddCssClass($"btn btn-{color} btn-{size}");
        button.Attributes["type"] = "submit";
        button.InnerHtml.Append(text);
        return button;
    }

    // Custom Label Helper with asterisk for required fields
    public static IHtmlContent RequiredLabel(
        this IHtmlHelper htmlHelper,
        string fieldName,
        string displayText)
    {
        var label = new TagBuilder("label");
        label.AddCssClass("fw-bold");
        label.InnerHtml.AppendHtml($"{displayText} <span style='color:red'>*</span>");
        return label;
    }
}

```

Views/Home/Index.cshtml — Usage

```

@using YourApp.Helpers

<h2>Custom HTML Helper Demo</h2>
<form>
    @Html.RequiredLabel("Name", "Full Name")
    <input class="form-control mb-3" name="Name" />

    @Html.StyledButton("Submit Form", "success", "lg")
    @Html.StyledButton("Cancel", "danger", "lg")
</form>

```

Q3. Create a Login form using HTML Helpers with validation.

Theory: HTML Helpers like `Html.ValidationMessageFor()` and `Html.ValidationSummary()` display model validation errors. Combined with Data Annotations, they provide client and server-side validation.

Models/LoginModel.cs

```

using System.ComponentModel.DataAnnotations;

public class LoginModel
{
    [Required(ErrorMessage = "Username is required")]
    [StringLength(50, MinimumLength = 3, ErrorMessage = "3-50 characters")]
    public string Username { get; set; }

    [Required(ErrorMessage = "Password is required")]
    [DataType(DataType.Password)]
    [MinLength(6, ErrorMessage = "Minimum 6 characters")]
    public string Password { get; set; }
}

```

Controllers/AccountController.cs

```

public class AccountController : Controller
{
    [HttpGet]
    public IActionResult Login() => View(new LoginModel());

    [HttpPost]

```

```

public IActionResult Login(LoginModel model)
{
    if (!ModelState.IsValid)
        return View(model);

    if (model.Username == "admin" && model.Password == "admin123")
    {
        TempData["Message"] = "Login Successful!";
        return RedirectToAction("Dashboard");
    }
    ModelState.AddModelError("", "Invalid username or password.");
    return View(model);
}

public IActionResult Dashboard() => Content("Welcome to Dashboard!");
}

```

Views/Account/Login.cshtml

```

@model LoginModel
@{ ViewData["Title"] = "Login"; }

<div class="container" style="max-width:400px; margin-top:50px;">
    <h2 class="text-center">Login</h2>

    @Html.ValidationSummary(true, "", new { @class = "alert alert-danger" })

    @using (Html.BeginForm("Login", "Account", FormMethod.Post))
    {
        <div class="mb-3">
            <label>Username</label>
            @Html.TextBox("Username", null, new { @class="form-control" })
            @Html.ValidationMessage("Username", "", new { @class="text-danger small" })
        </div>
        <div class="mb-3">
            <label>Password</label>
            @Html.Password("Password", null, new { @class="form-control" })
            @Html.ValidationMessage("Password", "", new { @class="text-danger small" })
        </div>
        <button type="submit" class="btn btn-primary w-100">Login</button>
    }
</div>

```

Q4. Demonstrate different ways to generate links in ASP.NET Core MVC: `Html.ActionLink()`, `Url.Action()`, `Anchor Tag Helper`.

Theory: ASP.NET Core MVC provides three main ways to generate hyperlinks:

- **`Html.ActionLink()`** — Generates an anchor tag using `HtmlHelper`
- **`Url.Action()`** — Returns only the URL string (not the full anchor tag)
- **Anchor Tag Helper (`asp-*`)** — Clean Razor syntax using `asp-controller` and `asp-action` attributes

Views/Home/Links.cshtml

```

@{ ViewData["Title"] = "Link Generation Demo"; }

<h2>Different Ways to Generate Links</h2>

<h4>1. Html.ActionLink()</h4>
@Html.ActionLink("Go to Home", "Index", "Home")
@Html.ActionLink("Employee List", "Index", "Employee", null, new { @class="btn btn-primary" })
@Html.ActionLink("Details", "Details", "Product", new { id = 5 }, new { @class="btn btn-info" })

```

```

<hr/>
<h4>2. Url.Action()</h4>
<a href="@Url.Action("Index","Home")">Home via Url.Action</a><br/>
<a href="@Url.Action("Details","Product", new { id=10 })">Product Details</a><br/>
<p>URL Only: @Url.Action("About","Home")</p>

<hr/>
<h4>3. Anchor Tag Helper</h4>
<a asp-controller="Home" asp-action="Index">Home (Tag Helper)</a><br/>
<a asp-controller="Product" asp-action="Details" asp-route-id="5"
  class="btn btn-success">Product Details</a><br/>
<a asp-controller="Student" asp-action="Register"
  asp-route-course="BCA" class="btn btn-warning">BCA Students</a>

```

Q5. Create a form using Form Tag Helper and submit model data.

Theory: Form Tag Helper (asp-controller, asp-action) is the modern Razor way to create HTML forms. It generates the form's action URL, method, and anti-forgery token automatically. Combined with input tag helpers (asp-for), it creates strongly-typed forms.

Models/FeedbackModel.cs

```

public class FeedbackModel
{
    public string Name { get; set; }
    public string Email { get; set; }
    public string Subject { get; set; }
    public string Message { get; set; }
    public int Rating { get; set; }
}

```

Controllers/FeedbackController.cs

```

public class FeedbackController : Controller
{
    [HttpGet]
    public IActionResult Submit() => View(new FeedbackModel());

    [HttpPost]
    public IActionResult Submit(FeedbackModel model)
    {
        TempData["FeedbackMsg"] = $"Thank you {model.Name}! Rating: {model.Rating}/5";
        return RedirectToAction("Submit");
    }
}

```

Views/Feedback/Submit.cshtml

```

@model FeedbackModel
<h2>Feedback Form (Form Tag Helper)</h2>

@if(TempData["FeedbackMsg"] != null)
{
    <div class="alert alert-success">@TempData["FeedbackMsg"]</div>
}

<form asp-controller="Feedback" asp-action="Submit" method="post">
    <div class="mb-3">
        <label asp-for="Name"></label>
        <input asp-for="Name" class="form-control" />
    </div>
    <div class="mb-3">
        <label asp-for="Email"></label>

```

```

        <input asp-for="Email" class="form-control" type="email" />
    </div>
    <div class="mb-3">
        <label asp-for="Subject"></label>
        <input asp-for="Subject" class="form-control" />
    </div>
    <div class="mb-3">
        <label asp-for="Message"></label>
        <textarea asp-for="Message" class="form-control" rows="4"></textarea>
    </div>
    <div class="mb-3">
        <label asp-for="Rating">Rating (1-5)</label>
        <input asp-for="Rating" class="form-control" type="number" min="1" max="5" />
    </div>
    <button type="submit" class="btn btn-primary">Submit Feedback</button>
</form>

```

Q6. Use the Image Tag Helper to display images from wwwroot folder.

Theory: The **Image Tag Helper** (asp-append-version) enhances the standard img tag. It appends a cache-busting query string based on file hash, ensuring browsers load updated images when the file changes.

Views/Home/Gallery.cshtml

```

@{ ViewData["Title"] = "Image Tag Helper Demo"; }

<h2>Image Tag Helper Demo</h2>

@* Basic image from wwwroot/images/ *@


@* Image Tag Helper with cache-busting (asp-append-version) *@


@* Multiple images in a grid *@
<div class="row mt-3">
    <div class="col-md-4">
        
        <p class="text-center">Image 1</p>
    </div>
    <div class="col-md-4">
        
        <p class="text-center">Image 2</p>
    </div>
    <div class="col-md-4">
        
        <p class="text-center">Image 3</p>
    </div>
</div>

```

Note: Place images in wwwroot/images/ folder. asp-append-version='true' adds ?v=hash to prevent stale cache.

Q7. Create a navigation menu using Anchor Tag Helper.

Theory: Anchor Tag Helpers use **asp-controller**, **asp-action**, and **asp-route-*** attributes to generate navigation links. They automatically handle URL generation and are cleaner than `Html.ActionLink()`.

Views/Shared/_Layout.cshtml — Navigation Menu

```
<!DOCTYPE html>
<html>
<head>
  <title>@ViewData["Title"]</title>
  <link rel="stylesheet" href="~/lib/bootstrap/css/bootstrap.min.css" />
</head>
<body>

<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" asp-controller="Home" asp-action="Index">
      Sangola Mahavidyalaya
    </a>
    <button class="navbar-toggler" type="button"
      data-bs-toggle="collapse" data-bs-target="#navMenu">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navMenu">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item">
          <a class="nav-link" asp-controller="Home" asp-action="Index">Home</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" asp-controller="Student" asp-action="Index">Students</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" asp-controller="Employee" asp-action="Index">Employees</a>
        </li>
        <li class="nav-item dropdown">
          <a class="nav-link dropdown-toggle" data-bs-toggle="dropdown">Courses</a>
          <ul class="dropdown-menu">
            <li>
              <a class="dropdown-item" asp-controller="Course"
                asp-action="Details" asp-route-id="1">BCA</a>
            </li>
            <li>
              <a class="dropdown-item" asp-controller="Course"
                asp-action="Details" asp-route-id="2">BSc CS</a>
            </li>
          </ul>
        </li>
        <li class="nav-item">
          <a class="nav-link" asp-controller="Account" asp-action="Login">Login</a>
        </li>
      </ul>
    </div>
  </div>
</nav>

<div class="container mt-4">@RenderBody()</div>

<script src="~/lib/bootstrap/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Q8. Create and use a Partial Tag Helper for reusable UI components.

Theory: The **Partial Tag Helper** (<partial name='...' />) is the modern, preferred way to render partial views. It is cleaner than Html.Partial() and supports passing model data via the model attribute.

Views/Shared/_AlertMessage.cshtml

```
@model string
@if (!string.IsNullOrEmpty(Model))
{
    <div class="alert alert-info alert-dismissible fade show">
        <strong>Info:</strong> @Model
        <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
    </div>
}
```

Views/Shared/_StudentCard.cshtml

```
@model YourApp.Models.StudentModel
<div class="card mb-3" style="max-width:400px;">
    <div class="card-header bg-primary text-white">
        Student ID: @Model.StudentId
    </div>
    <div class="card-body">
        <h5 class="card-title">@Model.Name</h5>
        <p class="card-text">Course: @Model.Course</p>
    </div>
</div>
```

Views/Home/Index.cshtml — Using Partial Tag Helper

```
@{
    var student = new YourApp.Models.StudentModel
    { StudentId=1, Name="Rahul Patil", Course="BCA" };
}

<h2>Partial Tag Helper Demo</h2>

@* Basic usage *@
<partial name="_AlertMessage" model="@("Welcome to Sangola Mahavidyalaya!")" />

@* With model *@
<partial name="_StudentCard" model="student" />

@* Partial from ViewData *@
<partial name="_AlertMessage" model='@(ViewData["Notice"] as string)' />
```

Q9. Create a Custom Tag Helper that formats text in uppercase.

Theory: Custom Tag Helpers extend HTML by creating new tags or enhancing existing ones. They inherit from **TagHelper** base class and override the **Process()** method. They must be registered in _ViewImports.cshtml.

TagHelpers/UpperCaseTagHelper.cs

```
using Microsoft.AspNetCore.Razor.TagHelpers;

// Targets <uppercase> tag in Razor views
[HtmlTargetElement("uppercase")]
public class UpperCaseTagHelper : TagHelper
{
    // Optional attribute: color
    public string Color { get; set; } = "black";

    public override async Task ProcessAsync(
        TagHelperContext context,
```



```

        TagHelperOutput output)
    {
        // Change tag to <span>
        output.TagName = "span";

        // Get inner content
        var content = await output.GetChildContentAsync();
        string originalText = content.GetContent();

        // Convert to uppercase
        output.Content.SetContent(originalText.ToUpper());

        // Apply color style
        output.Attributes.SetAttribute("style", $"color:{Color}; font-weight:bold;");
    }
}

```

Views/_ViewImports.cshtml — Register Tag Helper

```

@using YourApp
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
@addTagHelper *, YourApp

```

Views/Home/Index.cshtml — Usage

```
<h2>Custom Tag Helper Demo</h2>
```

```

<p>Normal text: hello world</p>
<p>Uppercase: <uppercase>hello world</uppercase></p>
<p>Colored: <uppercase color="blue">sangola mahavidyalaya</uppercase></p>
<p>Red: <uppercase color="red">asp.net core mvc</uppercase></p>

```

Q10. Implement Cache Tag Helper to cache a section of the page.

Theory: The **Cache Tag Helper** (<cache>) caches a portion of a Razor view in server memory. It supports expiry options like expires-after, expires-on, expires-sliding, and vary-by-* attributes.

Program.cs — Enable Memory Cache

```

builder.Services.AddControllersWithViews();
builder.Services.AddMemoryCache(); // Required for Cache Tag Helper

```

Views/Home/Index.cshtml

```
<h2>Cache Tag Helper Demo</h2>
```

```

@* Cache for 60 seconds *@
<cache expires-after="@TimeSpan.FromSeconds(60)">
    <p>Cached Time: @DateTime.Now.ToString("hh:mm:ss tt")</p>
    <p>This content will not change for 60 seconds.</p>
</cache>

@* Always updating (not cached) *@
<p>Live Time: @DateTime.Now.ToString("hh:mm:ss tt")</p>

@* Cache with sliding expiration (reset on each access) *@
<cache expires-sliding="@TimeSpan.FromMinutes(5)">
    <p>Sliding Cache: @DateTime.Now.ToString("hh:mm:ss tt")</p>
</cache>

@* Cache per user *@
<cache vary-by-user="true" expires-after="@TimeSpan.FromMinutes(10)">
    <p>Per-User Cache: @DateTime.Now</p>
</cache>

```

```
@* Cache with unique name *@  
<cache name="productListCache" expires-after="@TimeSpan.FromMinutes(2)">  
    <p>Product list cached at: @DateTime.Now</p>  
</cache>
```

Note: Cache Tag Helper requires AddMemoryCache() to be called in Program.cs services.

Q11. Create a model class `Employee_Registration` using Data Annotation attributes: `Required`, `StringLength`, `Range`, `EmailAddress`, `Compare`.

Theory: Data Annotations are attributes from `System.ComponentModel.DataAnnotations` namespace used to validate model properties. They work on both server-side (`ModelState.IsValid`) and client-side (with jQuery Validation).

Models/`Employee_Registration.cs`

```
using System.ComponentModel.DataAnnotations;
```

```
public class Employee_Registration
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Employee Name is required")]
    [StringLength(100, MinimumLength = 3,
        ErrorMessage = "Name must be between 3 and 100 characters")]
    [Display(Name = "Employee Name")]
    public string Name { get; set; }

    [Required(ErrorMessage = "Department is required")]
    [StringLength(50)]
    public string Department { get; set; }

    [Required(ErrorMessage = "Age is required")]
    [Range(18, 65, ErrorMessage = "Age must be between 18 and 65")]
    public int Age { get; set; }

    [Required(ErrorMessage = "Salary is required")]
    [Range(10000, 500000, ErrorMessage = "Salary must be between 10,000 and 5,00,000")]
    public decimal Salary { get; set; }

    [Required(ErrorMessage = "Email is required")]
    [EmailAddress(ErrorMessage = "Enter a valid email address")]
    public string Email { get; set; }

    [Required(ErrorMessage = "Phone is required")]
    [Phone(ErrorMessage = "Enter a valid phone number")]
    [StringLength(10, MinimumLength = 10, ErrorMessage = "Phone must be 10 digits")]
    public string PhoneNumber { get; set; }

    [Required(ErrorMessage = "Password is required")]
    [DataType(DataType.Password)]
    [StringLength(20, MinimumLength = 6,
        ErrorMessage = "Password must be 6-20 characters")]
    public string Password { get; set; }

    [Required(ErrorMessage = "Please confirm password")]
    [DataType(DataType.Password)]
    [Compare("Password", ErrorMessage = "Passwords do not match")]
    [Display(Name = "Confirm Password")]
    public string ConfirmPassword { get; set; }
}
```

Controllers/`EmployeeController.cs`

```
public class EmployeeController : Controller
{
    [HttpGet]
    public IActionResult Register() => View(new Employee_Registration());
}
```

```

[HttpPost]
public IActionResult Register(Employee_Registration model)
{
    if (!ModelState.IsValid)
        return View(model);

    TempData["Msg"] = $"Employee {model.Name} registered successfully!";
    return RedirectToAction("Register");
}
}

```

Views/Employee/Register.cshtml

```

@model Employee_Registration
<h2>Employee Registration</h2>
@Html.ValidationSummary(false, "", new { @class="alert alert-danger" })

<form asp-action="Register" method="post">
    <div class="mb-2">
        <label asp-for="Name"></label>
        <input asp-for="Name" class="form-control" />
        <span asp-validation-for="Name" class="text-danger"></span>
    </div>
    <div class="mb-2">
        <label asp-for="Age"></label>
        <input asp-for="Age" class="form-control" />
        <span asp-validation-for="Age" class="text-danger"></span>
    </div>
    <div class="mb-2">
        <label asp-for="Salary"></label>
        <input asp-for="Salary" class="form-control" />
        <span asp-validation-for="Salary" class="text-danger"></span>
    </div>
    <div class="mb-2">
        <label asp-for="Email"></label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
    </div>
    <div class="mb-2">
        <label asp-for="Password"></label>
        <input asp-for="Password" class="form-control" />
        <span asp-validation-for="Password" class="text-danger"></span>
    </div>
    <div class="mb-2">
        <label asp-for="ConfirmPassword"></label>
        <input asp-for="ConfirmPassword" class="form-control" />
        <span asp-validation-for="ConfirmPassword" class="text-danger"></span>
    </div>
    <button type="submit" class="btn btn-primary">Register</button>
</form>

```

Q12. Implement Custom Data Annotation to validate that the entered number must be EVEN only.

Theory: Custom Data Annotations are created by inheriting from **ValidationAttribute** and overriding the **IsValid()** method. They can be applied to model properties just like built-in annotations.

Validations/EvenNumberAttribute.cs

```

using System.ComponentModel.DataAnnotations;

public class EvenNumberAttribute : ValidationAttribute

```

```

{
    public EvenNumberAttribute()
    {
        ErrorMessage = "Only EVEN numbers are allowed.";
    }

    protected override ValidationResult IsValid(
        object value, ValidationContext validationContext)
    {
        if (value == null)
            return new ValidationResult("Value is required.");

        if (int.TryParse(value.ToString(), out int number))
        {
            if (number % 2 == 0)
                return ValidationResult.Success;
            else
                return new ValidationResult(ErrorMessage);
        }

        return new ValidationResult("Please enter a valid integer.");
    }
}

```

Models/NumberModel.cs

```

public class NumberModel
{
    [Required(ErrorMessage = "Number is required")]
    [EvenNumber]
    [Display(Name = "Enter a Number")]
    public int? Number { get; set; }
}

```

Controllers/NumberController.cs + View

```

// Controller
public class NumberController : Controller
{
    [HttpGet]
    public IActionResult Check() => View(new NumberModel());

    [HttpPost]
    public IActionResult Check(NumberModel model)
    {
        if (!ModelState.IsValid) return View(model);
        TempData["Result"] = $"{model.Number} is an EVEN number!";
        return RedirectToAction("Check");
    }
}

// View: Views/Number/Check.cshtml
@model NumberModel
<h2>Even Number Validator (Custom Data Annotation)</h2>
@if(TempData["Result"] != null)
{
    <div class="alert alert-success">@TempData["Result"]</div>
}
<form asp-action="Check" method="post">
    <label asp-for="Number"></label>
    <input asp-for="Number" class="form-control w-25" />
    <span asp-validation-for="Number" class="text-danger"></span>

```

```
<button type="submit" class="btn btn-primary mt-2">Check</button>
</form>
```

Q13. Implement Blacklist and Whitelist validation using Data Annotations.

Theory:

- **Blacklist Validation:** Rejects values from a forbidden list (e.g., banned words/usernames).
- **Whitelist Validation:** Only allows values from an approved list (e.g., valid country codes).

Validations/BlacklistAttribute.cs

```
public class BlacklistAttribute : ValidationAttribute
{
    private readonly string[] _blacklistedValues;

    public BlacklistAttribute(params string[] blacklistedValues)
    {
        _blacklistedValues = blacklistedValues;
        ErrorMessage = "This value is not allowed (blacklisted).";
    }

    protected override ValidationResult IsValid(
        object value, ValidationContext ctx)
    {
        if (value == null) return ValidationResult.Success;
        string input = value.ToString().ToLower();

        foreach (var item in _blacklistedValues)
        {
            if (input == item.ToLower())
                return new ValidationResult(
                    $"{value}' is a blacklisted value.");
        }
        return ValidationResult.Success;
    }
}
```

Validations/WhitelistAttribute.cs

```
public class WhitelistAttribute : ValidationAttribute
{
    private readonly string[] _allowedValues;

    public WhitelistAttribute(params string[] allowedValues)
    {
        _allowedValues = allowedValues;
        ErrorMessage = "Value not in approved list.";
    }

    protected override ValidationResult IsValid(
        object value, ValidationContext ctx)
    {
        if (value == null) return new ValidationResult("Value required.");
        string input = value.ToString().ToLower();

        foreach (var item in _allowedValues)
        {
            if (input == item.ToLower())
                return ValidationResult.Success;
        }
        return new ValidationResult(
```

```

        $"'{value}' is not in the allowed list: {string.Join(", ", _allowedValues)}");
    }
}

```

Models/UserModel.cs — Using both

```

public class UserModel
{
    [Required]
    [Blacklist("admin", "root", "superuser", "administrator")]
    [Display(Name = "Username")]
    public string Username { get; set; }

    [Required]
    [Whitelist("India", "USA", "UK", "Canada", "Australia")]
    [Display(Name = "Country")]
    public string Country { get; set; }
}

```

Q14. Demonstrate Display and Formatting attributes: Display(Name), DataType, DisplayFormat.

Theory:

- **[Display(Name)]** — Sets the label text shown in the view for a property
- **[DataType]** — Specifies the type (Password, Date, Email, Currency, etc.) for display purposes
- **[DisplayFormat]** — Formats how data is displayed (e.g., date format, currency, null display)

Models/ProductModel.cs

```

using System.ComponentModel.DataAnnotations;

public class ProductModel
{
    public int Id { get; set; }

    [Display(Name = "Product Name")]
    [Required]
    public string Name { get; set; }

    [Display(Name = "Unit Price (Rs.)")]
    [DataType(DataType.Currency)]
    [DisplayFormat(DataFormatString = "{0:C2}",
        ApplyFormatInEditMode = false)]
    public decimal Price { get; set; }

    [Display(Name = "Manufacturing Date")]
    [DataType(DataType.Date)]
    [DisplayFormat(DataFormatString = "{0:dd-MM-yyyy}",
        ApplyFormatInEditMode = true)]
    public DateTime ManufactureDate { get; set; }

    [Display(Name = "Expiry Date")]
    [DataType(DataType.Date)]
    [DisplayFormat(DataFormatString = "{0:dd/MM/yyyy}",
        NullDisplayText = "No Expiry")]
    public DateTime? ExpiryDate { get; set; }

    [Display(Name = "Product Description")]
    [DataType(DataType.MultilineText)]
    public string Description { get; set; }

    [Display(Name = "Website URL")]

```

```

[DataType(DataType.Url)]
public string Website { get; set; }

[Display(Name = "Contact Email")]
[DataType(DataType.EmailAddress)]
public string Email { get; set; }

[Display(Name = "Discount %")]
[DisplayFormat(DataFormatString = "{0:P0}",
    NullDisplayText = "No Discount")]
public double? Discount { get; set; }
}

```

Views/Product/Details.cshtml — Using display attributes

```

@model ProductModel
<h2>Product Details (Display & Format Attributes)</h2>
<table class="table table-bordered">
  <tr>
    <th>@Html.DisplayNameFor(m => m.Name)</th>
    <td>@Html.DisplayFor(m => m.Name)</td>
  </tr>
  <tr>
    <th>@Html.DisplayNameFor(m => m.Price)</th>
    <td>@Html.DisplayFor(m => m.Price)</td>
  </tr>
  <tr>
    <th>@Html.DisplayNameFor(m => m.ManufactureDate)</th>
    <td>@Html.DisplayFor(m => m.ManufactureDate)</td>
  </tr>
  <tr>
    <th>@Html.DisplayNameFor(m => m.ExpiryDate)</th>
    <td>@Html.DisplayFor(m => m.ExpiryDate)</td>
  </tr>
  <tr>
    <th>@Html.DisplayNameFor(m => m.Email)</th>
    <td>@Html.DisplayFor(m => m.Email)</td>
  </tr>
</table>

```


Q15. Demonstrate use of Cookies to store user state.

Theory: Cookies are small pieces of data stored on the client's browser. In ASP.NET Core, cookies are managed via **Response.Cookies** (write) and **Request.Cookies** (read). They are used to maintain state across requests.

Controllers/CookieController.cs

```
public class CookieController : Controller
{
    // Set Cookie
    public IActionResult SetCookie()
    {
        var options = new CookieOptions
        {
            Expires = DateTime.Now.AddDays(1),
            HttpOnly = true,
            IsEssential = true
        };
        Response.Cookies.Append("UserName", "Rahul Patil", options);
        Response.Cookies.Append("UserRole", "Student", options);
        Response.Cookies.Append("LastLogin",
            DateTime.Now.ToString("dd-MM-yyyy HH:mm"), options);

        TempData["Msg"] = "Cookies set successfully!";
        return RedirectToAction("ReadCookie");
    }

    // Read Cookie
    public IActionResult ReadCookie()
    {
        ViewBag.UserName = Request.Cookies["UserName"] ?? "Not Found";
        ViewBag.UserRole = Request.Cookies["UserRole"] ?? "Not Found";
        ViewBag.LastLogin = Request.Cookies["LastLogin"] ?? "Not Found";
        return View();
    }

    // Delete Cookie
    public IActionResult DeleteCookie()
    {
        Response.Cookies.Delete("UserName");
        Response.Cookies.Delete("UserRole");
        Response.Cookies.Delete("LastLogin");
        TempData["Msg"] = "All cookies deleted!";
        return RedirectToAction("ReadCookie");
    }
}
```

Views/Cookie/ReadCookie.cshtml

```
<h2>Cookie Demo</h2>
@if(TempData["Msg"] != null)
{
    <div class="alert alert-info">@TempData["Msg"]</div>
}
<table class="table table-bordered w-50">
    <tr><th>Username</th><td>@ViewBag.UserName</td></tr>
    <tr><th>Role</th><td>@ViewBag.UserRole</td></tr>
    <tr><th>Last Login</th><td>@ViewBag.LastLogin</td></tr>
</table>
<a asp-action="SetCookie" class="btn btn-primary">Set Cookies</a>
<a asp-action="DeleteCookie" class="btn btn-danger">Delete Cookies</a>
```

Q16. Implement Encrypted Cookies in ASP.NET Core.

Theory: ASP.NET Core uses **Data Protection API** to encrypt cookie values. The `IDataProtector` interface encrypts and decrypts sensitive data before storing in cookies, preventing tampering.

Program.cs

```
builder.Services.AddDataProtection(); // Add Data Protection
builder.Services.AddControllersWithViews();
```

Controllers/SecureCookieController.cs

```
using Microsoft.AspNetCore.DataProtection;

public class SecureCookieController : Controller
{
    private readonly IDataProtector _protector;

    public SecureCookieController(IDataProtectionProvider provider)
    {
        _protector = provider.CreateProtector("CookieProtection.v1");
    }

    // Set Encrypted Cookie
    public IActionResult SetSecureCookie()
    {
        string sensitiveData = "UserId=42|Role=Admin|Email=admin@college.com";

        // Encrypt the value
        string encryptedValue = _protector.Protect(sensitiveData);

        var options = new CookieOptions
        {
            Expires = DateTime.Now.AddHours(2),
            HttpOnly = true,
            Secure = true
        };
        Response.Cookies.Append("SecureUserData", encryptedValue, options);

        return RedirectToAction("ReadSecureCookie");
    }

    // Read and Decrypt Cookie
    public IActionResult ReadSecureCookie()
    {
        string encryptedValue = Request.Cookies["SecureUserData"];
        if (encryptedValue != null)
        {
            string decryptedValue = _protector.Unprotect(encryptedValue);
            ViewBag.DecryptedData = decryptedValue;
            ViewBag.EncryptedData = encryptedValue[..30] + "..."; // Show partial
        }
        else
        {
            ViewBag.DecryptedData = "No secure cookie found.";
        }
        return View();
    }
}
```

Views/SecureCookie/ReadSecureCookie.cshtml

```
<h2>Encrypted Cookie Demo</h2>
```

```

<p><b>Encrypted (stored in browser):</b><br/>
    <code>@ViewBag.EncryptedData</code></p>
<p><b>Decrypted (actual data):</b><br/>
    <span class="text-success">@ViewBag.DecryptedData</span></p>
<a asp-action="SetSecureCookie" class="btn btn-warning">Set Encrypted Cookie</a>

```

Q17. Implement Session Management to store user login details.

Theory: Sessions store data on the server side. Each client gets a unique session ID stored in a cookie. ASP.NET Core uses ISession interface with AddSession() and UseSession() middleware.

Program.cs

```

builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});
builder.Services.AddControllersWithViews();

var app = builder.Build();
app.UseSession(); // Must be before MapControllerRoute
app.UseRouting();
app.MapControllerRoute("default", "{controller=Home}/{action=Index}/{id?}");
app.Run();

```

Controllers/SessionController.cs

```

public class SessionController : Controller
{
    // Login and store session
    [HttpGet]
    public IActionResult Login() => View();

    [HttpPost]
    public IActionResult Login(string username, string password)
    {
        if (username == "admin" && password == "1234")
        {
            HttpContext.Session.SetString("UserName", username);
            HttpContext.Session.SetString("UserRole", "Admin");
            HttpContext.Session.SetInt32("UserId", 1);
            HttpContext.Session.SetString("LoginTime",
                DateTime.Now.ToString("dd-MM-yyyy HH:mm:ss"));
            return RedirectToAction("Dashboard");
        }
        ViewBag.Error = "Invalid Credentials";
        return View();
    }

    // Dashboard
    public IActionResult Dashboard()
    {
        ViewBag.UserName = HttpContext.Session.GetString("UserName");
        ViewBag.Role = HttpContext.Session.GetString("UserRole");
        ViewBag.UserId = HttpContext.Session.GetInt32("UserId");
        ViewBag.LoginTime = HttpContext.Session.GetString("LoginTime");

        if (ViewBag.UserName == null)
            return RedirectToAction("Login");
    }
}

```

```

        return View();
    }

    // Logout
    public IActionResult Logout()
    {
        HttpContext.Session.Clear();
        return RedirectToAction("Login");
    }
}

```

Q18. Create functionality to Upload a Single File.

Theory: File upload in ASP.NET Core uses **IFormFile** interface. The form must have **enctype='multipart/form-data'**. Files are saved using **FileStream** or **CopyToAsync()**.

Controllers/FileController.cs

```

using Microsoft.AspNetCore.Mvc;

public class FileController : Controller
{
    private readonly IWebHostEnvironment _env;

    public FileController(IWebHostEnvironment env)
    {
        _env = env;
    }

    [HttpGet]
    public IActionResult Upload() => View();

    [HttpPost]
    public async Task<IActionResult> Upload(IFormFile file)
    {
        if (file == null || file.Length == 0)
        {
            ViewBag.Error = "Please select a file.";
            return View();
        }

        // Save to wwwroot/uploads/
        string uploadsFolder = Path.Combine(_env.WebRootPath, "uploads");
        Directory.CreateDirectory(uploadsFolder);

        string uniqueFileName = Guid.NewGuid().ToString() + "_" + file.FileName;
        string filePath = Path.Combine(uploadsFolder, uniqueFileName);

        using (var stream = new FileStream(filePath, FileMode.Create))
        {
            await file.CopyToAsync(stream);
        }

        ViewBag.FileName = uniqueFileName;
        ViewBag.Size = file.Length;
        ViewBag.Type = file.ContentType;
        return View();
    }
}

```

Views/File/Upload.cshtml

```

<h2>Single File Upload</h2>
@if(ViewBag.Error != null)
{
    <div class="alert alert-danger">@ViewBag.Error</div>
}
@if(ViewBag.FileName != null)
{
    <div class="alert alert-success">
        File uploaded: @ViewBag.FileName<br/>
        Size: @ViewBag.Size bytes | Type: @ViewBag.Type
    </div>
}
<form asp-action="Upload" method="post" enctype="multipart/form-data">
    <div class="mb-3">
        <label>Select File:</label>
        <input type="file" name="file" class="form-control" />
    </div>
    <button type="submit" class="btn btn-primary">Upload</button>
</form>

```

Q19. Recreate file upload with restriction on uploaded file size (max 2MB).

Controllers/FileController.cs — Size Restricted

```

[HttpPost]
public async Task<IActionResult> UploadWithSizeLimit(IFormFile file)
{
    if (file == null || file.Length == 0)
    {
        ViewBag.Error = "Please select a file.";
        return View("Upload");
    }

    // Check file size: 2MB = 2 * 1024 * 1024 = 2097152 bytes
    long maxFileSize = 2 * 1024 * 1024;
    if (file.Length > maxFileSize)
    {
        ViewBag.Error = $"File size exceeds 2MB limit. " +
            $"Your file: {file.Length / 1024} KB";
        return View("Upload");
    }

    string uploadsFolder = Path.Combine(_env.WebRootPath, "uploads");
    Directory.CreateDirectory(uploadsFolder);
    string uniqueFileName = Guid.NewGuid() + "_" + file.FileName;
    string filePath = Path.Combine(uploadsFolder, uniqueFileName);

    using (var stream = new FileStream(filePath, FileMode.Create))
        await file.CopyToAsync(stream);

    ViewBag.Success = $"File '{file.FileName}' uploaded! Size: {file.Length/1024} KB";
    return View("Upload");
}

```

Program.cs — Set max request size (optional)

```

builder.Services.Configure<FormOptions>(options =>
{
    options.MultipartBodyLengthLimit = 2 * 1024 * 1024; // 2MB server-side limit
});

```

Q20. Recreate file upload with restriction on uploaded file type (only .jpg, .png, .pdf).

Controllers/FileController.cs — File Type Restricted

```
[HttpPost]
public async Task<IActionResult> UploadWithTypeCheck(IFormFile file)
{
    if (file == null || file.Length == 0)
    {
        ViewBag.Error = "Please select a file.";
        return View("Upload");
    }

    // Allowed extensions
    var allowedExtensions = new[] { ".jpg", ".jpeg", ".png", ".pdf" };
    string ext = Path.GetExtension(file.FileName).ToLower();

    if (!allowedExtensions.Contains(ext))
    {
        ViewBag.Error = $"Invalid file type '{ext}'. " +
            "Only .jpg, .png, .pdf are allowed.";
        return View("Upload");
    }

    // Also check MIME type
    var allowedMimeTypes = new[]
    {
        "image/jpeg", "image/png", "application/pdf"
    };
    if (!allowedMimeTypes.Contains(file.ContentType))
    {
        ViewBag.Error = "Invalid file content type.";
        return View("Upload");
    }

    string uploadsFolder = Path.Combine(_env.WebRootPath, "uploads");
    Directory.CreateDirectory(uploadsFolder);
    string uniqueFileName = Guid.NewGuid() + "_" + file.FileName;
    string filePath = Path.Combine(uploadsFolder, uniqueFileName);

    using (var stream = new FileStream(filePath, FileMode.Create))
        await file.CopyToAsync(stream);

    ViewBag.Success = $"File '{file.FileName}' uploaded successfully!";
    return View("Upload");
}
```

Q21. Create functionality to Upload Multiple Files and display them.

Controllers/FileController.cs — Multiple Files

```
[HttpGet]
public IActionResult UploadMultiple() => View();

[HttpPost]
public async Task<IActionResult> UploadMultiple(List<IFormFile> files)
{
    if (files == null || files.Count == 0)
    {
        ViewBag.Error = "Please select at least one file.";
        return View();
    }

    string uploadsFolder = Path.Combine(_env.WebRootPath, "uploads");
```

```

Directory.CreateDirectory(uploadsFolder);

var uploadedFiles = new List<string>();

foreach (var file in files)
{
    if (file.Length > 0)
    {
        string uniqueName = Guid.NewGuid() + "_" + file.FileName;
        string filePath = Path.Combine(uploadsFolder, uniqueName);

        using var stream = new FileStream(filePath, FileMode.Create);
        await file.CopyToAsync(stream);

        uploadedFiles.Add(uniqueName);
    }
}

ViewBag.UploadedFiles = uploadedFiles;
ViewBag.Count = uploadedFiles.Count;
return View();
}

```

Views/File/UploadMultiple.cshtml

```

<h2>Multiple File Upload</h2>
<form asp-action="UploadMultiple" method="post" enctype="multipart/form-data">
    <div class="mb-3">
        <label>Select Files (multiple):</label>
        <input type="file" name="files" class="form-control" multiple />
    </div>
    <button type="submit" class="btn btn-primary">Upload All</button>
</form>

@if(ViewBag.UploadedFiles != null)
{
    <h4 class="mt-3">Uploaded @ViewBag.Count file(s):</h4>
    <div class="row">
        @foreach(string fileName in ViewBag.UploadedFiles)
        {
            string ext = System.IO.Path.GetExtension(fileName).ToLower();
            <div class="col-md-3 mb-3">
                @if(ext == ".jpg" || ext == ".jpeg" || ext == ".png")
                {
                    
                }
                else
                {
                    <div class="border p-3 text-center">
                        <i class="bi bi-file-earmark"></i>
                        <p>@fileName</p>
                    </div>
                }
            </div>
        }
    </div>
}
</div>
}

```

Q22. Export data from database to Excel File.

Theory: Excel export in ASP.NET Core uses the **EPPlus** or **ClosedXML** NuGet package. Data from model/database is written to an Excel workbook and returned as a **FileResult**.

Install NuGet Package

```
dotnet add package ClosedXML
```

Controllers/ExcelController.cs

```
using ClosedXML.Excel;
using Microsoft.AspNetCore.Mvc;

public class ExcelController : Controller
{
    // Sample data (replace with DB data)
    private List<Employee> GetEmployees() => new List<Employee>
    {
        new Employee{Id=1, Name="Rahul Patil", Dept="IT", Salary=55000},
        new Employee{Id=2, Name="Sneha More", Dept="HR", Salary=42000},
        new Employee{Id=3, Name="Amit Kumar", Dept="Finance", Salary=60000},
    };

    public IActionResult ExportToExcel()
    {
        var employees = GetEmployees();

        using var workbook = new XLWorkbook();
        var sheet = workbook.Worksheets.Add("Employees");

        // Header row
        sheet.Cell(1, 1).Value = "ID";
        sheet.Cell(1, 2).Value = "Name";
        sheet.Cell(1, 3).Value = "Department";
        sheet.Cell(1, 4).Value = "Salary";

        // Style header
        var headerRow = sheet.Row(1);
        headerRow.Style.Font.Bold = true;
        headerRow.Style.Fill.BackgroundColor = XLColor.DarkBlue;
        headerRow.Style.Font.FontColor = XLColor.White;

        // Data rows
        int row = 2;
        foreach (var emp in employees)
        {
            sheet.Cell(row, 1).Value = emp.Id;
            sheet.Cell(row, 2).Value = emp.Name;
            sheet.Cell(row, 3).Value = emp.Dept;
            sheet.Cell(row, 4).Value = emp.Salary;
            row++;
        }

        // Auto-fit columns
        sheet.Columns().AdjustToContents();

        // Return as file download
        using var stream = new MemoryStream();
        workbook.SaveAs(stream);
        stream.Position = 0;
        return File(stream.ToArray(),
```



```

        "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        "Employees.xlsx");
    }
}

```

View — Export Button

```

<a asp-controller="Excel" asp-action="ExportToExcel"
  class="btn btn-success">Export to Excel</a>

```

Q23. Generate a PDF report from Model Class data.

Theory: PDF generation in ASP.NET Core uses **iTextSharp** or **DinkToPdf** or **QuestPDF** library. Data from model is formatted and returned as a PDF file download.

Install NuGet Package

```
dotnet add package itext7
```

Controllers/PdfController.cs

```

using iText.Kernel.Pdf;
using iText.Layout;
using iText.Layout.Element;
using iText.Layout.Properties;
using iText.Kernel.Colors;

public class PdfController : Controller
{
    public IActionResult GeneratePdf()
    {
        var employees = new List<Employee>
        {
            new Employee{Id=1,Name="Rahul Patil",Dept="IT",Salary=55000},
            new Employee{Id=2,Name="Sneha More",Dept="HR",Salary=42000},
            new Employee{Id=3,Name="Amit Kumar",Dept="Finance",Salary=60000},
        };

        using var ms = new MemoryStream();
        var writer = new PdfWriter(ms);
        var pdf = new PdfDocument(writer);
        var doc = new Document(pdf);

        // Title
        doc.Add(new Paragraph("Employee Report")
            .SetFontSize(18)
            .SetBold()
            .SetTextAlignment(TextAlignment.CENTER));

        doc.Add(new Paragraph($"Generated: {DateTime.Now:dd-MM-yyyy}")
            .SetFontSize(10)
            .SetTextAlignment(TextAlignment.CENTER));

        doc.Add(new Paragraph(" "));

        // Table
        var table = new iText.Layout.Element.Table(4)
            .UseAllAvailableWidth();

        // Header cells
        string[] headers = { "ID", "Name", "Department", "Salary" };
        foreach (var h in headers)
        {

```

```

        table.AddHeaderCell(new Cell()
            .Add(new Paragraph(h).SetBold())
            .SetBackgroundColor(ColorConstants.DARK_GRAY)
            .SetFontColor(ColorConstants.WHITE));
    }

    // Data rows
    foreach (var emp in employees)
    {
        table.AddCell(emp.Id.ToString());
        table.AddCell(emp.Name);
        table.AddCell(emp.Dept);
        table.AddCell(emp.Salary.ToString("C"));
    }

    doc.Add(table);
    doc.Close();

    return File(ms.ToArray(), "application/pdf", "EmployeeReport.pdf");
}
}

```

Q24. Create an application to implement full CRUD operations for Student entity (Insert, Select, Update, Delete).

Theory: CRUD (Create, Read, Update, Delete) with Entity Framework Core involves setting up a DbContext, model class, and controller with EF Core operations like Add(), Find(), Update(), Remove(), and SaveChanges().

Install EF Core

```

dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools

```

Models/Student.cs

```

using System.ComponentModel.DataAnnotations;

public class Student
{
    public int Id { get; set; }

    [Required]
    [StringLength(100)]
    public string Name { get; set; }

    [Required]
    public string Course { get; set; }

    [Range(1, 100)]
    public int Marks { get; set; }

    [EmailAddress]
    public string Email { get; set; }
}

```

Data/AppDbContext.cs

```

using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }
}

```

```

        public DbSet<Student> Students { get; set; }
    }

```

Program.cs — Register DbContext

```

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

```

appsettings.json

```

{
  "ConnectionStrings": {
    "DefaultConnection":
      "Server=(localdb)\mssqllocaldb;Database=StudentDB;Trusted_Connection=True;"
  }
}

```

Controllers/StudentController.cs — Full CRUD

```

public class StudentController : Controller
{
    private readonly AppDbContext _db;
    public StudentController(AppDbContext db) { _db = db; }

    // SELECT – List all
    public IActionResult Index()
    {
        var students = _db.Students.ToList();
        return View(students);
    }

    // INSERT – GET
    public IActionResult Create() => View();

    // INSERT – POST
    [HttpPost]
    public IActionResult Create(Student student)
    {
        if (!ModelState.IsValid) return View(student);
        _db.Students.Add(student);
        _db.SaveChanges();
        return RedirectToAction("Index");
    }

    // UPDATE – GET
    public IActionResult Edit(int id)
    {
        var student = _db.Students.Find(id);
        if (student == null) return NotFound();
        return View(student);
    }

    // UPDATE – POST
    [HttpPost]
    public IActionResult Edit(Student student)
    {
        if (!ModelState.IsValid) return View(student);
        _db.Students.Update(student);
        _db.SaveChanges();
        return RedirectToAction("Index");
    }
}

```

```
// DELETE
public IActionResult Delete(int id)
{
    var student = _db.Students.Find(id);
    if (student == null) return NotFound();
    _db.Students.Remove(student);
    _db.SaveChanges();
    return RedirectToAction("Index");
}
}
```

Migration Commands

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

Q25. Create an ASP.NET Core MVC application to demonstrate Data Annotation Attributes in EF Core.

Theory: Data Annotation attributes in EF Core are used to configure the database schema directly on model classes. They control table names, column names, primary keys, foreign keys, nullability, and constraints.

Models/Department.cs + Employee.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

[Table("Departments")] // Table name override
public class Department
{
    [Key] // Primary Key
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int DeptId { get; set; }

    [Required]
    [Column("DepartmentName")] // Column name override
    [StringLength(100)]
    public string Name { get; set; }

    public ICollection<Employee> Employees { get; set; }
}

[Table("Employees")]
public class Employee
{
    [Key]
    public int EmpId { get; set; }

    [Required(ErrorMessage = "Name required")]
    [Column("EmployeeName", TypeName = "nvarchar(100)")]
    [MaxLength(100)]
    public string Name { get; set; }

    [Column(TypeName = "decimal(10,2)")]
    [Range(5000, 999999)]
    public decimal Salary { get; set; }

    [EmailAddress]
    [Index(IsUnique = true)] // Unique index
    public string Email { get; set; }

    [NotMapped] // Not stored in DB
```

```
public string FullDisplayName => $"{Name} ({Email})";

[Timestamp] // Concurrency token
public byte[] RowVersion { get; set; }

[ForeignKey("Department")] // Foreign Key
public int DeptId { get; set; }

public Department Department { get; set; }
}
```

Note: [NotMapped] skips property in DB. [Timestamp] adds row versioning for concurrency control. [Index] creates DB index.

Q26. Create an ASP.NET Core MVC application to demonstrate Code First Approach in EF Core.

Theory: In the **Code First** approach, you define C# model classes first, and EF Core generates the database schema from those classes using Migrations. This is the most common approach in modern ASP.NET Core development.

Steps: Define Models → Create DbContext → Configure Connection String → Add Migration → Update Database

Step 1: Install Packages

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Step 2: Models/Product.cs

```
using System.ComponentModel.DataAnnotations;

public class Product
{
    public int Id { get; set; }

    [Required]
    [StringLength(150)]
    public string Name { get; set; }

    [Required]
    public string Category { get; set; }

    [Range(1, 999999)]
    public decimal Price { get; set; }

    public int Stock { get; set; }

    public DateTime CreatedDate { get; set; } = DateTime.Now;
}
```

Step 3: Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }

    public DbSet<Product> Products { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Seed data
        modelBuilder.Entity<Product>().HasData(
            new Product { Id=1, Name="Laptop", Category="Electronics",
                          Price=55000, Stock=10 },
            new Product { Id=2, Name="Mouse", Category="Accessories",
                          Price=500, Stock=50 }
        );
    }
}
```

Step 4: appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection":
```

```

        "Server=(localdb)\mssqllocaldb;Database=ProductDB;Trusted_Connection=True;"
    }
}

```

Step 5: Program.cs

```

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

```

Step 6: Migrations

```

dotnet ef migrations add CreateProductsTable
dotnet ef database update
# Database is now created from C# code!

```

Step 7: Controllers/ProductController.cs

```

public class ProductController : Controller
{
    private readonly AppDbContext _db;
    public ProductController(AppDbContext db) { _db = db; }

    public IActionResult Index() => View(_db.Products.ToList());

    [HttpGet] public IActionResult Create() => View();

    [HttpPost]
    public IActionResult Create(Product p)
    {
        if (!ModelState.IsValid) return View(p);
        _db.Products.Add(p);
        _db.SaveChanges();
        return RedirectToAction("Index");
    }
}

```

Q27. Create an ASP.NET Core MVC application to demonstrate DB First Approach in EF Core.

Theory: In the **Database First** approach, the database already exists. EF Core **Scaffolding** tool (dotnet ef dbcontext scaffold) generates C# model classes and DbContext from the existing database automatically. Useful when working with legacy databases.

Steps: Create DB → Scaffold Models → Use Generated Context in App

Step 1: Create Database in SQL Server

```

-- Run in SQL Server Management Studio (SSMS)
CREATE DATABASE CollegeDB;
GO
USE CollegeDB;

CREATE TABLE Students (
    Id INT PRIMARY KEY IDENTITY,
    Name NVARCHAR(100) NOT NULL,
    Course NVARCHAR(50),
    Marks INT,
    Email NVARCHAR(100)
);

INSERT INTO Students VALUES ('Rahul Patil', 'BCA', 85, 'rahul@gmail.com');
INSERT INTO Students VALUES ('Sneha More', 'BSc CS', 90, 'sneha@gmail.com');

```

Step 2: Install Required Packages

```

dotnet add package Microsoft.EntityFrameworkCore.SqlServer

```

```
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Step 3: Scaffold (Reverse Engineer) the Database

```
dotnet ef dbcontext scaffold "Server=(localdb)\mssqllocaldb;Database=CollegeDB;Trusted_Connection=True;" Mic
```

This command generates **Models/Student.cs** and **Data/CollegeDbContext.cs** automatically from the existing database.

Step 4: Generated CollegeDbContext.cs (auto-generated)

```
// Auto-generated by EF Core Scaffolding
public partial class CollegeDbContext : DbContext
{
    public CollegeDbContext(DbContextOptions<CollegeDbContext> options)
        : base(options) { }

    public virtual DbSet<Student> Students { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Student>(entity =>
        {
            entity.Property(e => e.Name).HasMaxLength(100);
            entity.Property(e => e.Course).HasMaxLength(50);
            entity.Property(e => e.Email).HasMaxLength(100);
        });
    }
}
```

Step 5: Program.cs — Register Scaffolded Context

```
builder.Services.AddDbContext<CollegeDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection"));
```

Step 6: Controllers/StudentController.cs

```
public class StudentController : Controller
{
    private readonly CollegeDbContext _db;
    public StudentController(CollegeDbContext db) { _db = db; }

    // Uses existing DB – no migrations needed
    public IActionResult Index()
    {
        return View(_db.Students.ToList());
    }
}
```

Code First vs DB First Comparison:

- Code First: Define C# models → EF creates DB | Best for new projects
- DB First: DB exists → EF generates C# models | Best for existing/legacy databases